# Direct Preference Optimization (DPO)

## CS 5624: Natural Language Processing

*Fall 2026*

https://tuvllms.github.io/nlp-fall-2026/

**Tu Vu**

VIRGINIA TECH

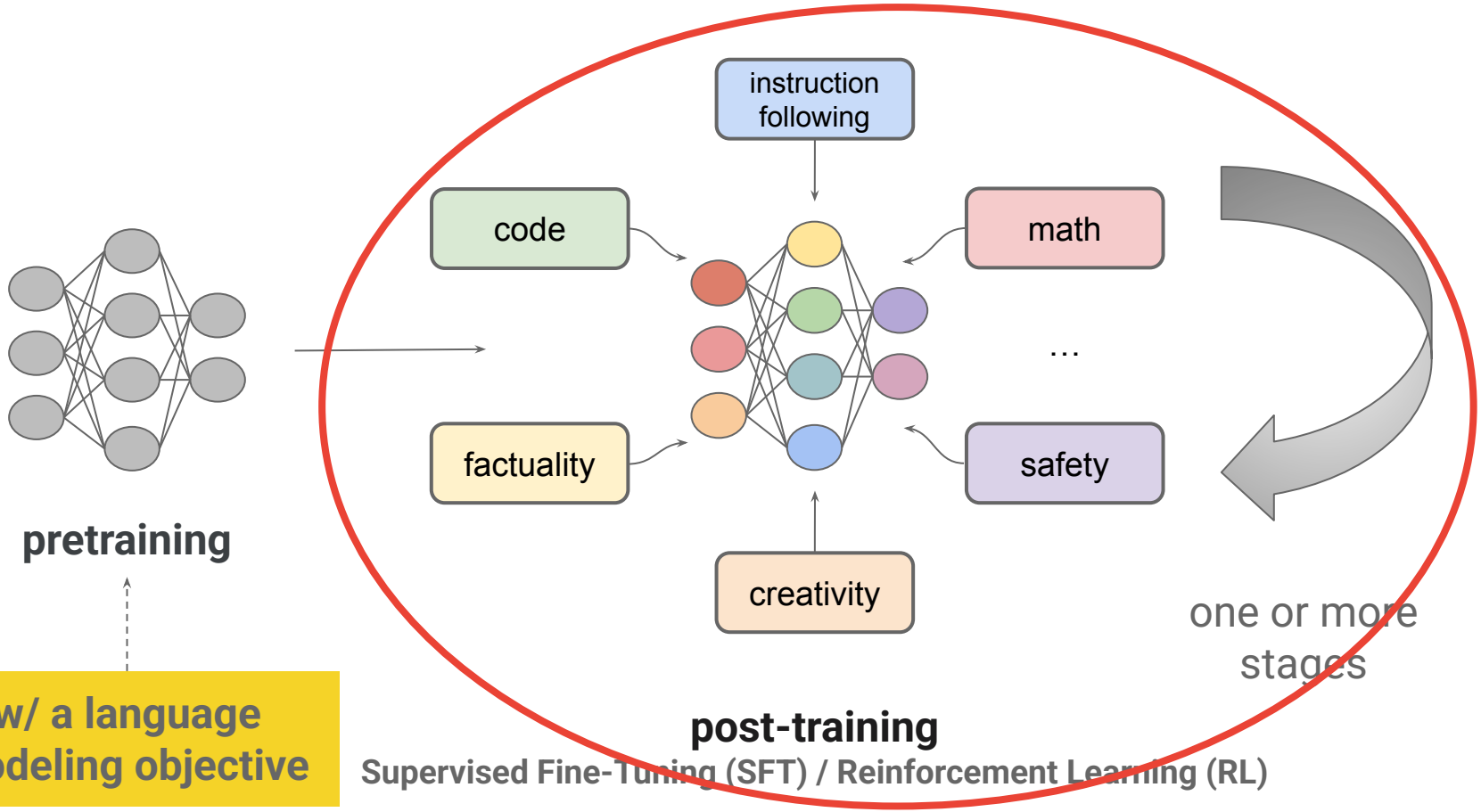
# Logistics

- Final projects
  - Project proposal
    - Template available on Piazza @23, due on 10/12 (no late days)
    - Potential topics shared on Piazza @24
  - Sample project available on Piazza @21

- HW1 due tomorrow Friday 10/9

# AI news

# Where are we?


instruction following
code
math
...
factuality
safety
creativity
one or more stages
pretraining
w/ a language modeling objective
post-training
Supervised Fine-Tuning (SFT) / Reinforcement Learning (RL)

# Reinforcement learning from human feedback (RLHF)

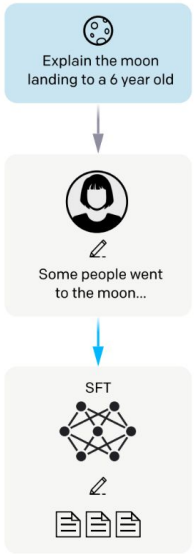
Step 1

**Collect demonstration data, and train a supervised policy.**

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



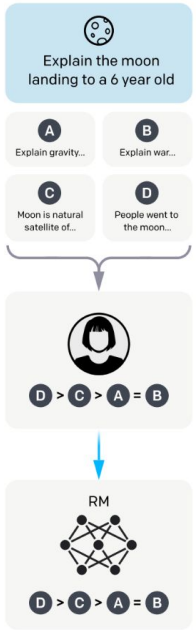

Step 2

**Collect comparison data, and train a reward model.**

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.




Step 3

**Optimize a policy against the reward model using reinforcement learning.**

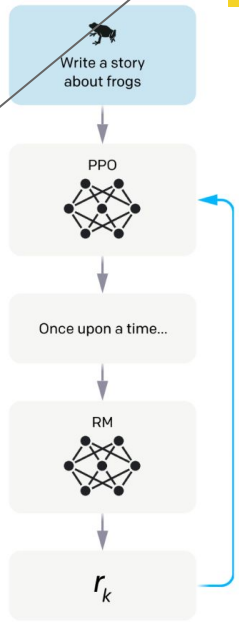
**Learn from data generated by the current policy**

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



# Step 3: RL training

**The second term prevents the model from deviating too far from the distribution on which the reward model is accurate.**

**$y \sim \pi_\theta(y|x)$ rollout**

$$\max_{\pi_\theta} \mathbb{E}_{x\sim D, y\sim\pi_\theta(y|x)} \left[r_\phi(x,y)\right] - \beta D_{KL}\left[\pi_\theta(y|x) \,||\, \pi_{\text{ref}}(y|x)\right]$$

where $\beta$ is a parameter controlling the deviation from the base reference policy $\pi_{\text{ref}}$, namely the initial SFT model $\pi^{SFT}$. In practice, the language model policy $\pi_\theta$ is also initialized to $\pi^{SFT}$.

# Expectation

The **expectation** (or expected value) of a random variable $X$ is

$$\boxed{\mathbb{E}[X] = \sum_x x\, P(X = x)}$$

where:

- $x$ is a possible value of $X$
- $P(X = x)$ is the probability of that value
- $\sum_x$ means "sum over all possible values of $X$"

So expectation is simply a **probability-weighted average**.

**Example: fair six-sided die**

Let $X$ be the number rolled. Each outcome has probability $1/6$:

$$\begin{aligned}\mathbb{E}[X] &= 1\left(\frac{1}{6}\right) + 2\left(\frac{1}{6}\right) + \cdots + 6\left(\frac{1}{6}\right) \\ &= \frac{1+2+3+4+5+6}{6} \\ &= \boxed{3.5}.\end{aligned}$$

This does **not** mean you can roll a $3.5$. It means that the average roll approaches $3.5$ over many rolls.

**An expectation is a weighted average over all possible outcomes, where each outcome is weighted by how likely it is.**

# KL divergence

Assume two different distributions for predicting the next word:

- $P$ (from Model 1):
  - *mat* → 0.7
  - *floor* → 0.2
  - *chair* → 0.1
- $Q$ (from Model 2):
  - *mat* → 0.5
  - *floor* → 0.3
  - *chair* → 0.2

### Kullback–Leibler (KL) Divergence Calculation

KL divergence measures how much $P$ diverges from $Q$:

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Substituting the values:

$$D_{KL}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.2 \log \frac{0.2}{0.3} + 0.1 \log \frac{0.1}{0.2}$$

# Proximal Policy Optimization (PPO)

In SFT, the token-level loss is

$$\mathcal{L}_{\mathrm{SFT}} = -\sum_{t} \log \pi_\theta(y_t \mid y_{<t}, x).$$

You can think of every demonstrated token as having weight $1$.

In vanilla policy-gradient RL, the corresponding objective is approximately

$$\mathcal{L}_{\mathrm{RL}} = -\sum_{t} A_t \log \pi_\theta(a_t \mid a_{<t}, x),$$

where $a_t$ is the sampled token and $A_t$ is its advantage. So the gradient looks like a **weighted NLL gradient**:

$$\nabla_\theta \mathcal{L}_{\mathrm{RL}} = -\sum_{t} A_t \nabla_\theta \log \pi_\theta(a_t \mid a_{<t}, x).$$

There is one important distinction from SFT. **SFT trains on target tokens supplied by the dataset, whereas RL usually trains on tokens sampled from the policy itself.**

**SFT**

The 5 tokens come from a **human/reference response**:

$$o = (o_1, \ldots, o_5) \sim \text{human data}.$$

We maximize

$$\boxed{J_{\text{SFT}}(\theta) = \frac{1}{5} \sum_{t=1}^{5} \log \pi_\theta(o_t \mid o_{<t})}$$

So the model is told:

$$\text{“Make the human's tokens more likely.”}$$

**RL**

The 5 tokens are **sampled from the model**:

$$\boxed{o = (o_1, \ldots, o_5) \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)}$$

Then we evaluate the generated response, get reward $r$, and derive advantage $A$.

Conceptually, the objective is

$$\boxed{J_{\text{RL}}(\theta) = \frac{1}{5} \sum_{t=1}^{5} A \log \pi_\theta(o_t \mid x, o_{<t})}$$

Thus:

$$A > 0 \quad \Rightarrow \quad \text{make the model's sampled tokens more likely}$$

and

$$A < 0 \quad \Rightarrow \quad \text{make the model's sampled tokens less likely.}$$

| Method | $G_t$ | $b_t$ | Advantage shape |
|---|---|---|---|
| REINFORCE | $R$ | batch mean | same for all tokens |
| RLOO | $R$ | group mean | same for all tokens |
| PPO | $R$ | $V(s_t)$ | different per token |

- **REINFORCE** → $A_t = G_t - b$
- **RLOO** → $A_t = R - \text{leave-one-out baseline}$
- **PPO** → $A_t$ from value function / GAE + clipped ratio

$$A_t = G_t - b_t$$

$$= R_{\text{shaped}} - b_t$$

$$= R_{\text{model}} - \beta \sum_t \log \frac{P_t(a_t)}{Q_t(a_t)} - b_t$$

# REINFORCE

$$b_t = \text{mean reward over batch.}$$

So if your batch has rewards $[R_1, R_2, R_3]$,

$$b_t = \frac{R_1 + R_2 + R_3}{3}.$$

This baseline is the same for all tokens in that sequence.

Then:

$$G_t - b_t = R - \text{batch mean.}$$

So every token in that completion gets the same advantage.

# RLOO

## Motivation

REINFORCE is noisy because the baseline is crude (often a batch average). RLOO improves the baseline by making it **prompt-conditional**.

## Setup

- For one prompt $x$, sample $K$ completions $y^{(1)}, \dots, y^{(K)}$.
- Score each completion: $R^{(k)} = R(x, y^{(k)})$.

## Baseline

For completion $k$, the baseline is the mean reward of the *other* completions:

$$b^{(k)} = \frac{1}{K-1} \sum_{i \neq k} R^{(i)}.$$

## Advantage

$$A^{(k)} = R^{(k)} - b^{(k)}.$$

# PPO

$$A_t = G_t - V(s_t)$$

# Monte Carlo

Assume

$$r_1 = \cdots = r_{T-1} = 0, \qquad r_T = R, \qquad \gamma = 1.$$

$$A_1 = R - V(s_1)$$

$$A_2 = R - V(s_2)$$

$$A_3 = R - V(s_3)$$

$$\vdots$$

$$A_{T-1} = R - V(s_{T-1})$$

$$A_T = R - V(s_T)$$

$$A_t = R - V(s_t), \qquad t = 1, \ldots, T.$$

$$V_\phi(s_t) \approx \mathbb{E}_\pi[G_t \mid s_t]$$

$$A_t = \underbrace{G_t}_{\text{what actually happened}} - \underbrace{\mathbb{E}[G_t \mid s_t]}_{\text{what we expected}}$$

## Step 1: What is $G_t$?

**The return is all future rewards:**

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \cdots$$

**Pull out the first reward:**

$$G_t = r_{t+1} + \gamma \left( r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \cdots \right)$$

**Look at the part inside the parentheses:**

$$\underbrace{r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \cdots}_{G_{t+1}}$$

**So exactly:**

$$G_t = r_{t+1} + \gamma G_{t+1}$$

### Step 2: But we don't know $G_{t+1}$ yet

At time $t$, the future hasn't happened yet.

That's where the value function comes in. Remember:

$$V(s_{t+1}) \approx \mathbb{E}[G_{t+1} \mid s_{t+1}]$$

**In words:**

$V(s_{t+1})$ predicts the return that we'll get starting from the next state.

Therefore, we approximate

$$G_{t+1} \approx V(s_{t+1})$$

and consequently:

$$G_t \approx r_{t+1} + \gamma V(s_{t+1})$$

Now plug this into our advantage equation:

$$A_t = G_t - V(s_t)$$

giving

$$A_t \approx r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

which is the **TD error**.

### In our LLM RL setting

We use

$$r_{t+1} = 0, \qquad \gamma = 1$$

so the TD error simplifies to

$$A_t \approx V(s_{t+1}) - V(s_t)$$

In words, the advantage measures **how much the value increases from the current state to the next state**.

# One-step Temporal-Difference (TD)

$$A_1 \approx V(s_2) - V(s_1)$$

$$A_2 \approx V(s_3) - V(s_2)$$

$$A_3 \approx V(s_4) - V(s_3)$$

$$\vdots$$

$$A_{T-1} \approx V(s_T) - V(s_{T-1})$$

$$A_T \approx R - V(s_T)$$

$$A_t \approx \begin{cases} V(s_{t+1}) - V(s_t), & t < T, \\ R - V(s_T), & t = T. \end{cases}$$

# Generalized Advantage Estimation (GAE)

with all weights $= 1$

$$A_1 \approx [V(s_2) - V(s_1)] + [V(s_3) - V(s_2)] + \cdots + [R - V(s_T)]$$

$$A_2 \approx [V(s_3) - V(s_2)] + [V(s_4) - V(s_3)] + \cdots + [R - V(s_T)]$$

$$A_3 \approx [V(s_4) - V(s_3)] + \cdots + [R - V(s_T)]$$

$$\vdots$$

$$A_{T-1} \approx [V(s_T) - V(s_{T-1})] + [R - V(s_T)]$$

$$A_T \approx R - V(s_T)$$

$$A_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots + (\gamma\lambda)^{T-t}\delta_T$$

Since the terms cancel,

$$A_1 \approx R - V(s_1)$$

$$A_2 \approx R - V(s_2)$$

$$A_3 \approx R - V(s_3)$$

$$\vdots$$

$$A_T \approx R - V(s_T)$$

Thus, when all GAE weights are $1$,

$$A_t^{\text{GAE}} \approx R - V(s_t).$$

So in this setting,

$$\text{GAE with weights } 1 = \text{Full Rollout / Monte Carlo.}$$

# Reward model vs. Value model

## 1. The reward model learns a function of full sequences

Reward model:

$$R(x, y)$$

Input:

```
(prompt, full completion)
```

Example:

| input | reward |
|---|---|
| (Hello, "Bonjour le monde") | 0.9 |
| (Hello, "Salut le chien") | 0.1 |

It **never sees prefixes**.

# Reward model vs. Value model (cont'd)

## 2. The value model sees prefixes

Value model input:

$$s_t = (x, a_1, \ldots, a_{t-1})$$

Examples during training:

| prefix | label $G_t$ |
|---|---|
| prompt | 0.9 |
| prompt + Bonjour | 0.9 |
| prompt + Bonjour le | 0.9 |

But across many trajectories the same prefix can lead to **different final rewards**.

Example dataset:

# Reward model vs. Value model (cont'd)

| prefix | completion | reward |
|---|---|---|
| Bonjour | Bonjour le monde | 0.9 |
| Bonjour | Bonjour le chien mange une voiture | 0.1 |

So the value model learns

$$V(\text{Bonjour}) \approx 0.5$$

which is the **expected reward of continuations under the policy**.

## 3. The reward model cannot compute this expectation

The reward model only knows

$$R(x, y)$$

for a **specific finished sequence**.

It does not model

$$\mathbb{E}_{\pi}[R(x, y) \mid s_t]$$

which is what the value function must approximate.

---

## 4. The value function depends on the policy

This expectation depends on the **current policy**.

As the policy improves:

```
prefix → better completions
```

so

```
expected reward increases
```

The value model must be **updated continuously**.

The reward model stays fixed.

## 5. Another practical reason

Reward models are usually:

- trained with preference ranking loss
- not calibrated to produce consistent numeric expectations

Value models are trained with **regression to returns**, which matches the RL objective.

---

## Intuition

Think of it like writing an essay.

Reward model:

> "How good is this finished essay?"

Value model:

> "Given what is written so far, how good do I expect the final essay to be?"

These are different predictions.

---

## Key takeaway

Even though the **same final reward supervises the value model**, the value function learns the **expected reward of partial prefixes under the current policy**, which the reward model cannot represent.

# PPO (stable updates + token-level credit)

## A. PPO idea #1: do not move the policy too far (the ratio + clipping)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \exp\left(\log \pi_\theta - \log \pi_{\text{old}}\right).$$

PPO optimizes a clipped objective that prevents $\rho_t$ from leaving $[1-\epsilon,\ 1+\epsilon]$ in a way that would create overly large updates.

- $\pi_{\text{old}}$ is the policy that generated the batch.
- You can do multiple gradient steps on the same batch because importance ratios correct for drift.
- Clipping creates a trust region that stabilizes training.

# Core PPO policy objective (clipped)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

The PPO clipped objective is:

$$\boxed{L^{\text{policy}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\,A_t\right)\right]}$$

This is what we maximize.

Let

$$\epsilon = 0.2, \qquad A_t = 2$$

and

$$\pi_{\text{old}} = 0.5, \qquad \pi_\theta = 0.8.$$

Then

$$r_t = \frac{0.8}{0.5} = 1.6$$

Without clip:

$$r_t A_t = 1.6(2) = 3.2$$

With clip:

$$\text{clip}(1.6, 0.8, 1.2)(2) = 1.2(2) = 2.4$$

Therefore,

$$L_t^{\text{PPO}} = \min(3.2, 2.4) = 2.4.$$

Let

$$\epsilon = 0.2, \qquad A_t = 2, \qquad r_t = 0.6$$

Original:

$$r_t A_t = 0.6(2) = 1.2$$

Clipped:

$$\text{clip}(0.6, 0.8, 1.2)(2) = 0.8(2) = 1.6$$

Therefore,

$$L_t^{\text{PPO}} = \min(1.2, 1.6) = 1.2$$

Initially,

$$\pi_\theta = \pi_{\text{old}}$$

so

$$r_t = 1$$

and

$$L_t = r_t A_t = A_t.$$

More importantly, even though $r_t = 1$, its **gradient is not zero**:

$$\nabla_\theta r_t = r_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

At $r_t = 1$,

$$\nabla_\theta L_t = A_t \nabla_\theta \log \pi_\theta(a_t|s_t) \neq 0.$$

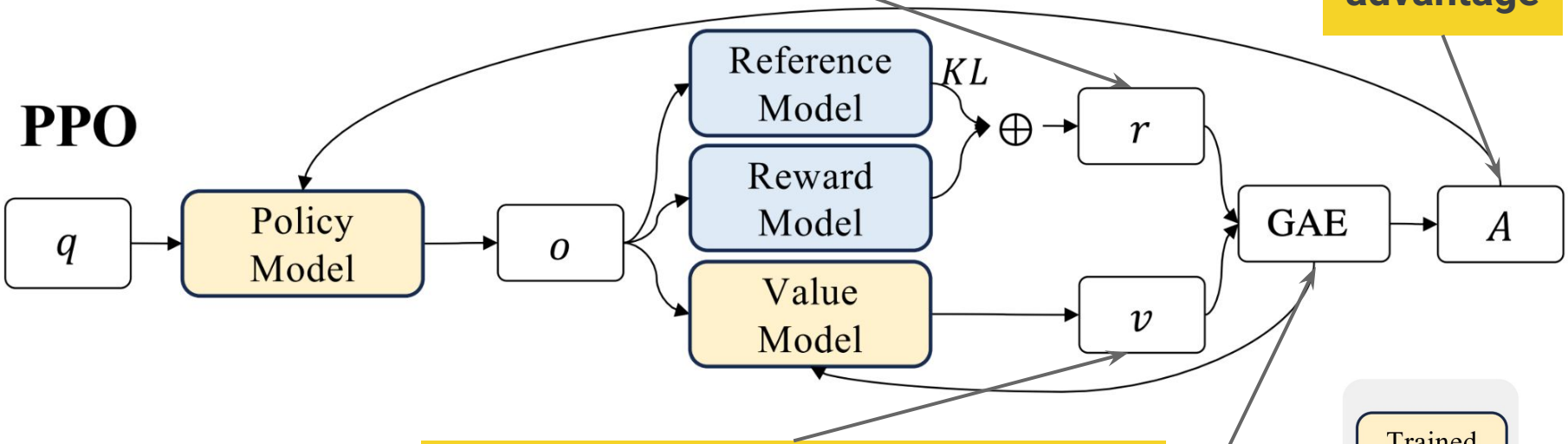
So the first gradient step changes $\pi_\theta$. Then

$$\pi_\theta \neq \pi_{\text{old}}$$

# PPO

Think of the reward model as a teacher who grades each essay you write

advantage




The critic is like your own internal expectation ("I usually get an 80 on essays like this")

GAE: Generalized Advantage Estimation

# Alignment techniques

- **SFT**: Supervised fine-tuning
- **RLHF / PPO**: Reinforcement Learning from Human Feedback / Proximal Policy Optimization
- **DPO**: Direct Preference Optimization
- **GRPO**: Group Relative Policy Optimization
- **DAPO**: **D**ecoupled Clip and **D**ynamic s**A**mpling **P**olicy **O**ptimization

**Green:** Non-RL

**Red: RL**

# DPO vs. RLHF

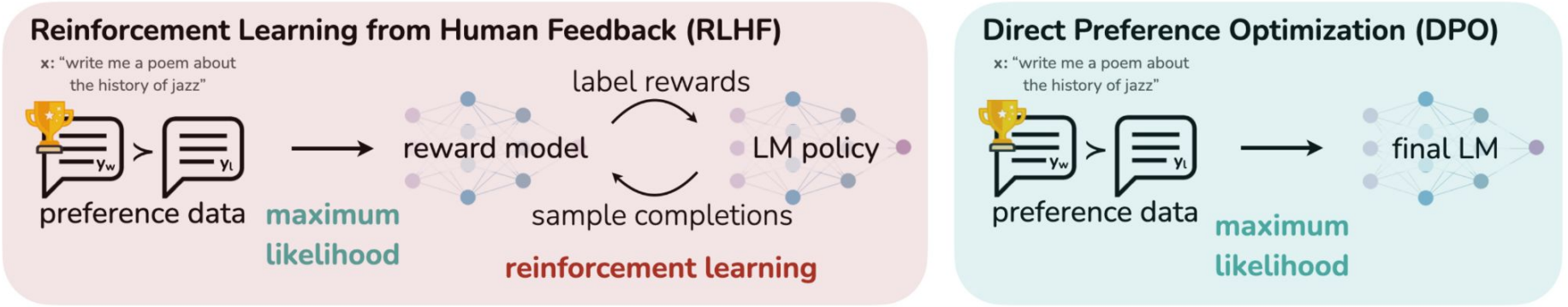



Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

# Direct Preference Optimization: Your Language Model is Secretly a Reward Model

**Rafael Rafailov**[*†] **Archit Sharma**[*†] **Eric Mitchell**[*†]

**Stefano Ermon**[†‡] **Christopher D. Manning**[†] **Chelsea Finn**[†]

[†]Stanford University [‡]CZ Biohub
{rafailov,architsh,eric.mitchell}@cs.stanford.edu

# Logarithms rules

1.

$$\log(A \cdot B) = \log(A) + \log(B)$$

The logarithm of a product is the sum of the logarithms.

2.

$$\log\left(\frac{A}{B}\right) = \log(A) - \log(B)$$

The logarithm of a quotient is the difference of the logarithms.

3.

$$\log(\exp(x)) = x$$

The logarithm of an exponential is simply the exponent.

4.

$$\log(A \cdot B \cdot C) = \log(A) + \log(B) + \log(C)$$

The logarithm of a product is the sum of the logarithms.

5.

$$\log\left(\frac{A \cdot B}{C}\right) = \log(A) + \log(B) - \log(C)$$

The logarithm of a product divided by a number is the sum of the logarithms of the numerator minus the logarith of the denominator.

6.

$$\log\left(\frac{A}{B \cdot C}\right) = \log(A) - \log(B) - \log(C)$$

The logarithm of a fraction with a product in the denominator is the logarithm of the numerator minus the sum of the logarithms of the denominator terms.

# Direct Preference Optimization (DPO)

$$\max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[ r(x,y) \right] - \beta D_{\mathrm{KL}} \left[ \pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x) \right]$$

# Minimization form

$$\max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[r(x,y)\right] - \beta D_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right]$$

$$\max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[\beta\left(\frac{1}{\beta} r(x,y) - D_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right]\right)\right]$$

$$= \max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[\frac{1}{\beta} r(x,y) - D_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right]\right]$$

$$= \max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[-\left(D_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right] - \frac{1}{\beta} r(x,y)\right)\right]$$

$$= \min_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[D_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right] - \frac{1}{\beta} r(x,y)\right]$$

# DPO objective

$$\min_{\pi} \mathbb{E}_{x \sim D, y \sim \pi(y|x)} \left[ D_{\mathrm{KL}} \left[ \pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x) \right] - \frac{1}{\beta} r(x, y) \right]$$

The **expectation** of a function $f(y)$ under a probability distribution $P(y)$ is defined as:

$$\mathbb{E}_{y \sim P(y)}[f(y)] = \sum_{y} P(y) f(y)$$

In words: **expectation is just a weighted sum, where $P(y)$ is the weight for each $f(y)$.**

For example, if we take expectation of $\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$ under $\pi(y|x)$:

$$\mathbb{E}_{y \sim \pi(y|x)} \left[ \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

this expands to:

$$\sum_{y} \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$$

which is exactly the KL divergence formula! $D_{\text{KL}} \left[ \pi(y|x) \,||\, \pi_{\text{ref}}(y|x) \right]$

# Recap: KL divergence

Assume two different distributions for predicting the next word:

- $P$ (from Model 1):
  - *mat* → 0.7
  - *floor* → 0.2
  - *chair* → 0.1
- $Q$ (from Model 2):
  - *mat* → 0.5
  - *floor* → 0.3
  - *chair* → 0.2

### Kullback–Leibler (KL) Divergence Calculation

KL divergence measures how much $P$ diverges from $Q$:

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Substituting the values:

$$D_{KL}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.2 \log \frac{0.2}{0.3} + 0.1 \log \frac{0.1}{0.2}$$

# DPO objective

$$\min_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[ D_{\mathrm{KL}} \left[ \pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x) \right] - \frac{1}{\beta} r(x,y) \right]$$

$$= \min_{\pi} \mathbb{E}_{x\sim D} \mathbb{E}_{y\sim\pi(y|x)} \left[ \log \frac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)} - \frac{1}{\beta} r(x,y) \right]$$

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y)$$

$$= \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \log \left( \exp \left( \frac{1}{\beta} r(x, y) \right) \right) + \log Z(x) - \log Z(x)$$

$$= \log \frac{Z(x)\pi(y|x)}{\pi_{\text{ref}}(y|x) \exp \left( \frac{1}{\beta} r(x, y) \right)} - \log Z(x)$$

$$= \log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left( \frac{1}{\beta} r(x, y) \right)} - \log Z(x)$$

# Softmax function

For a vector $y = [y_1, y_2, \ldots, y_V]$ of dimension $V$, the softmax transformation is calculated as:

$$\text{softmax}(y) = \left[ \frac{e^{y_1}}{\sum e^y}, \frac{e^{y_2}}{\sum e^y}, \ldots, \frac{e^{y_V}}{\sum e^y} \right]$$

where $\sum e^y = e^{y_1} + e^{y_2} + \cdots + e^{y_V}$.

We can define the partition function

$$Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

We have a valid distribution

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y)$$

$$= \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \log \left( \exp \left( \frac{1}{\beta} r(x,y) \right) \right) + \log Z(x) - \log Z(x)$$

$$= \log \frac{Z(x)\pi(y|x)}{\pi_{\text{ref}}(y|x) \exp \left( \frac{1}{\beta} r(x,y) \right)} - \log Z(x)$$

$$= \log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left( \frac{1}{\beta} r(x,y) \right)} - \log Z(x)$$

$$= \log \frac{\pi(y|x)}{\pi^*(y|x)} - \log Z(x)$$

# DPO objective (cont'd)

$$= \min_{\pi} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi(y|x)} \left[ \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \right]$$

$$= \min_{\pi} \mathbb{E}_{x \sim D} \left[ \mathbb{E}_{y \sim \pi(y|x)} \left[ \log \frac{\pi(y|x)}{\pi^*(y|x)} \right] - \log Z(x) \right]$$

$$= \min_{\pi} \mathbb{E}_{x \sim D} \left[ D_{KL}(\pi(y|x) || \pi^*(y|x)) - \log Z(x) \right]$$

**Optimal solution (based on Gibbs's inequality)**

$$\pi(y|x) = \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left( \frac{1}{\beta} r(x, y) \right)$$

# DPO objective under the Bradley-Terry model

$$p^*(y_w \succ y_l \mid x) = \frac{\exp(r^*(x, y_w))}{\exp(r^*(x, y_w)) + \exp(r^*(x, y_l))}$$

$$= \sigma\left(r^*(x, y_w) - r^*(x, y_l)\right)$$

# DPO objective under the Bradley-Terry model (cont'd)

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)$$

$$\log \pi^*(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r^*(x, y) - \log Z(x)$$

$$\frac{1}{\beta} r^*(x, y) = \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \log Z(x)$$

$$r^*(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

# DPO objective under the Bradley-Terry model (cont'd)

$$p^*(y_w \succ y_l \mid x) = \frac{\exp(r^*(x, y_w))}{\exp(r^*(x, y_w)) + \exp(r^*(x, y_l))}$$

$$= \sigma\left(r^*(x, y_w) - r^*(x, y_l)\right)$$

$$= \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

# DPO objective under the Bradley-Terry model (cont'd)

$$L_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

# DPO objective under the Bradley-Terry model (cont'd)

The gradient with respect to the parameters $\theta$ can be written as:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) =$$
$$-\beta \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{higher weight when reward estimate is wrong}}\Big[\underbrace{\nabla_\theta \log \pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta \log \pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}\Big]\Big],$$

where $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the reward implicitly defined by the language model $\pi_\theta$ and reference model $\pi_{\text{ref}}$ (more in Section 5).

arXiv:2404.10719v3 [cs.CL] 10 Oct 2024

# Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study

**Shusheng Xu** [1] **Wei Fu** [1] **Jiaxuan Gao** [1] **Wenjie Ye** [2] **Weilin Liu** [2]
**Zhiyu Mei** [1] **Guangju Wang** [2] **Chao Yu** [*1] **Yi Wu** [*1 2 3]

## Abstract

Reinforcement Learning from Human Feedback (RLHF) is currently the most widely used method to align large language models (LLMs) with human preferences. Existing RLHF methods can be roughly categorized as either *reward-based* or *reward-free*. Novel applications such as ChatGPT and Claude leverage *reward-based* methods that first learn a reward model and apply actor-critic algorithms, such as Proximal Policy Optimization (PPO). However, in academic benchmarks, the state-of-the-art results are often achieved via *reward-free* methods, such as Direct Preference Optimization (DPO). *Is DPO truly superior to PPO? Why does PPO perform poorly on these benchmarks?* In this paper, we first conduct both theoretical and empirical

2023). To leverage the formidable capabilities of LLMs in practical applications, a growing amount of research has underscored the importance of aligning these models with human preferences (Agrawal et al., 2023; Kadavath et al., 2022; Shi et al., 2023; Liang et al., 2021; Sheng et al., 2019). Various methods have been developed for fine-tuning LLMs, with popular approaches including Supervised Fine-Tuning (SFT) (Peng et al., 2023) and Reinforcement Learning from Human Feedback (RLHF) (Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022). Typically, fine-tuning involves two phases: SFT to establish a base model, followed by RLHF for enhanced performance. SFT involves imitating high-quality demonstration data, while RLHF refines LLMs through preference feedback.

Within RLHF, two prominent approaches are *reward-based* and *reward-free* methods. Reward-based methods, pioneered by OpenAI (Ouyang et al., 2022; Ziegler et al., 2019; Stiennon et al., 2020), construct a reward model using pref-

# Preference Fine-Tuning of LLMs Should Leverage Suboptimal, On-Policy Data

**Fahim Tajwar**[1*], **Anikait Singh**[2*], **Archit Sharma**[2], **Rafael Rafailov**[2], **Jeff Schneider**[1], **Tengyang Xie**[4], **Stefano Ermon**[2], **Chelsea Finn**[2] **and Aviral Kumar**[3]

[*]Equal contributions (ordered via coin-flip), [1]Carnegie Mellon University, [2]Stanford University, [3]Google DeepMind, [4]UW-Madison

Learning from preference labels plays a crucial role in fine-tuning large language models. There are several distinct approaches for preference fine-tuning, including supervised learning, on-policy reinforcement learning (RL), and contrastive learning. Different methods come with different implementation tradeoffs and performance differences, and existing empirical findings present different conclusions, for instance, some results show that online RL is quite important to attain good fine-tuning results, while others find (offline) contrastive or even purely supervised methods sufficient. This raises a natural question: ***what kind of approaches are important for fine-tuning with preference data and why?*** In this paper, we answer this question by performing a rigorous analysis of a number of fine-tuning techniques on didactic and full-scale LLM problems. Our main finding is that, in general, approaches that use on-policy sampling or attempt to push down the likelihood on certain responses (i.e., employ a "negative gradient") outperform offline and maximum likelihood objectives. We conceptualize our insights and unify methods that use on-policy sampling or negative gradient under a notion of mode-seeking objectives for categorical distributions. Mode-seeking objectives are able to alter probability mass on specific bins of a categorical distribution at a fast rate compared to maximum likelihood, allowing them to relocate masses across bins more effectively. Our analysis prescribes actionable insights for preference fine-tuning of LLMs and informs how data should be collected for maximal improvement.

[cs.LG] 2 Jun 2024

# Thank you!